

agsoft2025 / agcloud

Q

+

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Watch 0

☆

☆ 0 stars 0 forks 0 watching Branches Activity Tags

Public repository

main

1 Branch

0 Tags

Go to file

T

Go to file

Add file

Code

 Venkatakuna	Initial setup	1c32b4d · 36 minutes ago	
 documents	Initial setup	36 minutes ago	
 README.md	Initial setup	36 minutes ago	

agcloud

A self-hosted, real-time voice and video calling platform built on WebRTC and LiveKit.

Status: Design phase — specifications complete, implementation pending.

What is agcloud?

agcloud is an open communications platform similar in scope to ZEGOCLOUD or Twilio Video, built on open-source infrastructure. It is designed to be self-hostable, cost-effective at scale, and operationally simple.

Core Capabilities (MVP)

- 1:1 voice and video calls
- Group calls (up to 50 participants)
- Screen sharing
- Call recording
- Mobile-friendly PWA
- Real-time presence and contact management
- Self-hostable end to end

Future Capabilities (Phase 2+)

- Native mobile SDKs (iOS, Android)
- Live streaming (1 → many)
- AI noise suppression
- Multi-region deployment
- Developer SDK + API platform

Documentation

This repository contains a complete set of design specifications. Read them in this order:

#	Document	Purpose
1	context-info.md	Original product brief and roadmap
2	Feasibility-Research.md	Deep analysis of feasibility, tech stack risks, alternatives
3	LiveKit-Internals.md	LiveKit signaling protocol, deployment, integration details
4	Design-Diagrams.md	Architecture diagrams (original vs adopted approach)
5	Backend-Specification.md	Node.js backend service spec — security, reliability, observability, scalability
6	Frontend-Specification.md	Web app spec with screen mockups, performance budgets, UX guidelines

Tech Stack

Adopted Stack

Layer	Technology	Notes
Media (signaling + SFU + TURN)	LiveKit (self-hosted)	All-in-one WebRTC infrastructure
Backend API	Node.js + Fastify + TypeScript	High-performance HTTP framework
Database (persistent)	MongoDB	Users, call history, CDRs
Database (real-time)	Redis	Presence, active calls, pub/sub
Frontend	SvelteKit + TypeScript + Vite	Smallest bundle, no Virtual DOM
Push Notifications	FCM (Android/Web) + APNs (iOS)	
Observability	Pino + Prometheus + OpenTelemetry	Logs, metrics, tracing
Reverse Proxy	Caddy (MVP) / Nginx (production)	TLS termination, routing
Container Runtime	Docker + Docker Compose (MVP) / Kubernetes (production)	

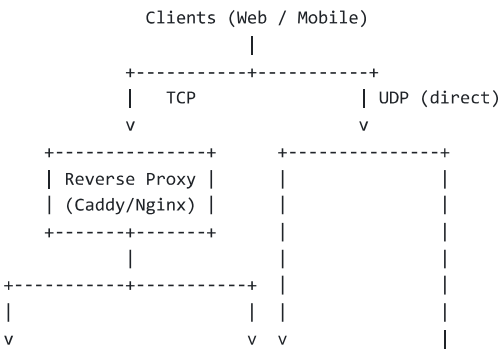
Why This Stack?

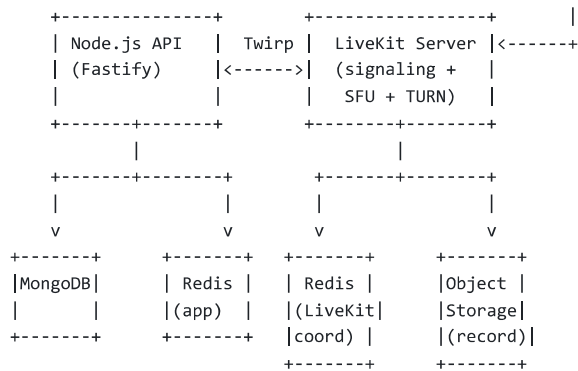
We evaluated the original mediasoup + Socket.IO + Coturn + MongoDB stack and pivoted to LiveKit because:

- LiveKit replaces three components (signaling, SFU, TURN) with a single service
- LiveKit ships maintained client SDKs across all platforms
- VC-funded company with full-time engineering team (vs 2-3 volunteer maintainers)
- Built-in clustering via Redis, recording via Egress, simulcast/dynacast
- MVP delivery: ~6 weeks vs ~16 weeks with self-built mediasoup orchestration

See [Feasibility-Research.md](#) for the full analysis.

Architecture Overview





Key rule: TCP traffic (HTTPS, WebSocket) routes through the proxy; UDP media (WebRTC) goes direct to LiveKit. Proxying UDP would add unacceptable latency.

See [Design-Diagrams.md](#) for detailed architecture diagrams.

Repository Structure

```
agcloud/
  README.md # This file
  context-info.md # Product brief
  Feasibility-Research.md # Tech feasibility analysis
  LiveKit-Internals.md # LiveKit deep-dive
  Design-Diagrams.md # Architecture diagrams
  Backend-Specification.md # Backend service spec
  Frontend-Specification.md # Web app spec

(planned)
  backend/ # Node.js + Fastify service
  frontend/ # SvelteKit web app
  infrastructure/
    docker-compose.yml # Local dev stack
    livekit/ # LiveKit config
    helm/ # Kubernetes charts (Phase 2)
  scripts/ # Dev tooling
```

Roadmap

Phase 1 — MVP (~6 weeks)

Week	Deliverable
1-2	Local dev stack (LiveKit + Node.js + MongoDB + Redis via Docker Compose). User auth, token generation, room creation, 1:1 audio/video calls
3-4	Group calls, mute/unmute, camera toggle, screen share, call history persistence, basic call quality monitoring
5-6	Recording (LiveKit Egress), call accept/reject flow with push notifications, basic admin dashboard, load testing

Phase 2 — Production (Months 2-3)

- Multi-AZ deployment (Kubernetes)
- Multi-region readiness
- Adaptive quality monitoring
- Comprehensive observability (Prometheus + Grafana + Tempo)
- Security hardening + penetration testing

Phase 3 — Platform (Months 4-6)

- Public REST API + webhooks
- SDK packages (JS, React Native)
- Developer documentation portal
- Billing + usage tracking

Phase 4 — Advanced (Months 6+)

- AI noise suppression
- Live streaming
- Native iOS/Android SDKs
- Advanced moderation tools

Getting Started

■ Note: Implementation has not started. The instructions below describe the planned developer experience.

📖 README



Prerequisites

- Node.js 20+
- pnpm 9+
- Docker + Docker Compose
- A modern browser (Chrome, Firefox, Edge, or Safari)

Local Development

```
# Clone the repository
git clone https://github.com/<your-org>/agcloud.git
cd agcloud

# Start the local stack (LiveKit, MongoDB, Redis)
docker-compose up -d

# Install backend dependencies and start
cd backend
pnpm install
cp .env.example .env
pnpm dev

# Install frontend dependencies and start (in a new terminal)
cd ../frontend
pnpm install
cp .env.example .env
pnpm dev

# Open the app
open http://localhost:5173
```



Environment Variables

Copy `.env.example` to `.env` in both `backend/` and `frontend/` directories and fill in:

- `MONGODB_URI`
- `REDIS_URL`
- `LIVEKIT_URL` / `LIVEKIT_API_KEY` / `LIVEKIT_API_SECRET`
- `JWT_SIGNING_KEY` (generate via `openssl rand -base64 64`)
- Push notification keys (FCM, APNs) — optional for local dev

See [Backend-Specification.md §9](#) for the full configuration reference.

Performance & Quality Targets

Backend

- API availability: **99.9%** ($\leq$ 43m downtime/month)
- API latency p99: **< 500ms**
- Call setup time p95: **< 2s**
- Push delivery: **95% within 10s**

Frontend

- First Contentful Paint: **< 1.0s** on 4G
- Time to Interactive: **< 2.5s** on 4G
- Initial JS bundle (gzipped): **< 80 KB**
- Lighthouse score: **> 95** across Performance, Accessibility, Best Practices, SEO

See [Backend-Specification.md §7](#) and [Frontend-Specification.md §3](#) for details.

Security

- TLS 1.2+ everywhere
- Argon2id password hashing
- JWT access tokens + rotating refresh tokens with reuse detection
- HTTP-only `secure` `sameSite=strict` refresh cookies
- Rate limiting (per-IP and per-user) via Redis
- Webhook signature verification (LiveKit)
- OWASP Top 10 mitigations documented per item
- Audit logging (90-day retention)

See [Backend-Specification.md §5](#) for the complete security spec.

Cost Expectations

Indicative monthly infrastructure costs (self-hosted on AWS/GCP):

Scale	Monthly Cost
MVP (50 concurrent calls)	\$750 - \$1,600
Small (500 concurrent)	\$7,000 - \$14,000
Medium (1,000 concurrent)	\$20,000 - \$34,000

Bandwidth is the dominant cost (~60-70% of total). See [Feasibility-Research.md §5](#) for the breakdown.

Contributing

This project is in early design phase. Once implementation begins, contribution guidelines will be added.

License

To be determined.

Acknowledgements

agcloud builds on the work of many open-source projects, including:

- [LiveKit](#) — WebRTC infrastructure
- [SvelteKit](#) — Frontend framework
- [Fastify](#) — Backend framework
- [MongoDB](#) and [Redis](#) — Data stores



Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1



Venkatakuna Venkata Kuna